

AMDP 練習 1：為什麼要 Code-to-Data + AMDP 架構總覽

Lecture

前面所有課程（基礎課 / OOP / REST）用的都是 **Open SQL**：`SELECT ... INTO TABLE` 把資料庫裡的資料整批搬到 ABAP 應用伺服器，再用 **LOOP** / 內表運算在應用層算邏輯。這個模式有個天生的成本：**資料要先搬過網路（資料庫伺服器 → 應用伺服器），才能開始算**——如果原始資料有幾百萬筆、但最後只要一個彙總數字（例如「各航空公司的航班數」），搬幾百萬筆資料只為了在應用層做一次 **COLLECT/SUM**，是很浪費的。

Code-to-Data（把邏輯搬到資料旁邊算，而不是把資料搬到邏輯旁邊算）是解決這個問題的思路：既然資料庫引擎本來就很擅長做集合運算（JOIN、聚合、排序），為什麼不讓運算邏輯直接在資料庫裡跑，只把「最後算完的結果」傳回 ABAP？這樣網路只需要傳輸「小小的最終結果」，不用傳輸「龐大的原始資料」。

AMDP（**ABAP Managed Database Procedure**）就是 SAP 讓 ABAP 開發者實踐 **Code-to-Data** 的機制：寫一個 ABAP Method，但這個 Method 的「實作內容」不是 ABAP 語句，而是一段 **SQLScript**（HANA 資料庫原生的程序語言）。這段 SQLScript 會被編譯成 HANA 資料庫裡真正的 Database Procedure，執行時直接在資料庫引擎裡跑，執行完只把結果（通常是一張或多張 Internal Table）傳回 ABAP 端。對呼叫端來說，AMDP Method 用起來跟一般 ABAP Method 一模一樣（**CALL METHOD** 或函數式呼叫），呼叫端完全不需要知道這個 Method 背後其實是資料庫在執行。

AMDP Method 的骨架：一個類別要實作 **IF_AMDP_MARKER_HDB** 這個空介面（純粹當「標記」用，告訴框架「這個類別裡可能有 AMDP Method」，介面本身沒有任何方法要實作），類別裡的某個 **CLASS-METHODS** 用 **METHOD ... BY DATABASE PROCEDURE FOR HDB LANGUAGE SQLSCRIPT** 宣告方法本體是 SQLScript，而不是普通 ABAP 語句。

學習目標

- 理解 Code-to-Data 的核心動機：把運算邏輯下推到資料庫引擎，減少「整批資料搬到應用層」的網路與記憶體成本
- 認識 AMDP 的骨架：**IF_AMDP_MARKER_HDB** 標記介面、**METHOD ... BY DATABASE PROCEDURE FOR HDB LANGUAGE SQLSCRIPT**、**OPTIONS READ-ONLY**（宣告這個程序不寫入資料庫）、**USING** <表/視圖>（宣告這段 SQLScript 會用到哪些資料庫物件，編譯期就要列清楚，不能臨時動態決定）
- 知道 **AMDP Method** 的簽章限制跟一般 **ABAP Method** 不一樣：所有 IMPORTING/EXPORTING 參數都必須用 **VALUE(...)** 宣告成傳值參數，不能是傳址參數；**DEFAULT** 值只能是常數/字面值，不能是 **sy-mandt** 這類系統欄位（這點跟一般 ABAP Method 允許 **DEFAULT sy-mandt** 不同）
- （本題最重要的觀念）理解 **AMDP** 完全不會自動處理 **Client (MANDT)** 過濾：Open SQL 的 **SELECT** 語句，ABAP 執行期框架會自動幫你加上 **WHERE mandt = sy-mandt**（這也是為什麼前面所有課程的 Open SQL 從來不用手動處理 client 欄位）；但 AMDP 的 SQLScript 是直接對 HANA 底層實體資料表操作，**沒有這層自動過濾**，**SELECT * FROM scarr** 撈到的是這張表裡「所有 Client」的資料，不是只有你目前登入的這個 Client

事前準備

ADT 端物件已由課程準備好（**\$TMP**）：

- **ZCL_AM01_CARRIERS**——AMDP 類別，**get_carriers** 方法用 SQLScript 查詢 **SCARR**
- **ZR_AM01_DEMO**——demo 程式，呼叫這個 AMDP Method 並用 **WRITE** 印出結果

題目需求 (對照已建好的答案物件，含開發過程實際踩到的坑)

1. **ZCL_AM01_CARRIERS** 的 AMDP Method 骨架：

```
``abap CLASS zcl_am01_carriers DEFINITION PUBLIC FINAL CREATE PUBLIC .
```

```
PUBLIC SECTION. INTERFACES if_amdp_marker_hdb.
```

```
TYPES: BEGIN OF ty_carrier, carrid TYPE s_carr_id, carrname TYPE s_carrname, END OF ty_carrier, tt_carrier TYPE STANDARD TABLE OF ty_carrier WITH EMPTY KEY.
```

```
CLASS-METHODS get_carriers IMPORTING VALUE(iv_mandt) TYPE mandt EXPORTING VALUE(et_carriers) TYPE tt_carrier.
```

```
ENDCLASS.
```

```
CLASS zcl_am01_carriers IMPLEMENTATION.
```

```
METHOD get_carriers BY DATABASE PROCEDURE FOR HDB LANGUAGE SQLSCRIPT OPTIONS READ-ONLY USING scarr.
```

```
et_carriers = SELECT carrid, carrname FROM scarr WHERE mandt = :iv_mandt ORDER BY carrid;
```

```
ENDMETHOD.
```

```
ENDCLASS. ``
```

這份程式碼是修過三次才能啟用的最終版本，過程中依序踩到三個 AMDP 特有的簽章限制 (一般 ABAP Method 不會遇到)：

- 第一次寫 **EXPORTING et_carriers TYPE tt_carrier (沒有 VALUE(...))** 啟用失敗：錯誤訊息「In an AMDP context, the parameter "ET_CARRIERS" ... cannot be defined as a reference parameter」——**AMDP Method** 的所有參數 (**IMPORTING** 跟 **EXPORTING** 都算) 必須用 **VALUE(...)** 宣告成傳值參數，一般 ABAP Method 的 **EXPORTING/IMPORTING** 預設就是傳址，AMDP 不允許這樣
- 加上 **VALUE(et_carriers)** 之後才發現另一個問題：一開始沒有 **iv_mandt** 這個參數，直接 **SELECT * FROM scarr (沒有 WHERE)** ——語法檢查/啟用都過，但實際執行後發現同一個航空公司代碼 (如 **AA**) 重複出現 6~8 次，見下方「這題最重要的發現」
- 補上 **IMPORTING iv_mandt TYPE mandt DEFAULT sy-mandt** 想讓呼叫端不用每次都傳 **client** 又啟用失敗：錯誤訊息「In an AMDP context, the default value of the parameter "IV_MANDT" ... must be a constant or literal」——**AMDP Method** 的 **DEFAULT** 值只能是常數或字面值，不能是 **sy-mandt** 這種系統欄位 (一般 ABAP Method 明確允許 **DEFAULT sy-mandt/sy-datum/sy-zeit/sy-langu** 這幾個系統欄位當預設值，AMDP 不允許)，最終拿掉 **DEFAULT**，改成呼叫端必須每次明確傳入 **iv_mandt = sy-mandt**

1. 這題最重要的發現：**AMDP** 不會幫你做 **Client** 過濾——開發過程中，**get_carriers** 一開始寫成不帶 **WHERE mandt = ...** 的版本：

```
``abap " 有問題的版本：沒有過濾 Client et_carriers = SELECT carrid, carrname FROM scarr
ORDER BY carrid; ``
```

實際呼叫後發現，SCARR 表裡同一個 carrid (例如 AA) 重複出現了 6 次。加上 MANDT 欄位一起撈出來診斷後，看到的是：

```
`` 100 AA American Airlines 110 AA American Airlines 130 AA American Airlines 135 AA
American Airlines 140 AA American Airlines 150 AA American Airlines ``
```

這套訓練系統裡，SCARR 這張表在 Client 000/100/110/130/135/140/150/400 都各自灌了一份一樣的示範資料——Open SQL 的 SELECT 語句，ABAP 執行期框架會自動幫每一句 SELECT 加上 WHERE mandt = sy-mandt，所以前面所有課程的 Open SQL 從來感覺不到這件事在發生；但 AMDP 的 SQLScript 是直接對 HANA 底層的實體資料表操作，這個自動過濾機制完全不存在，SELECT * FROM scarr 撈到的是這張表裡所有 Client 的資料。修正方式：把「目前登入的 Client 是誰」當成一個 IMPORTING 參數傳進去，SQLScript 裡自己手動加 WHERE mandt = :iv_mandt。

1. ZR_AM01_DEMO 呼叫端：

```
``abap REPORT zr_am01_demo.
```

```
zcl_am01_carriers=>get_carriers( EXPORTING iv_mandt = sy-mandt IMPORTING et_carriers = DATA(lt_carriers)
).
```

```
LOOP AT lt_carriers ASSIGNING FIELD-SYMBOL(<ls_carrier>). WRITE: / <ls_carrier>-carrid, <ls_carrier>-
carrname. ENDLOOP. ``
```

呼叫端寫法跟呼叫一般 ABAP Method 完全一樣（函數式呼叫、EXPORTING/IMPORTING 語法），呼叫端完全不需要知道 get_carriers 背後其實是一段跑在資料庫引擎裡的 SQLScript 程序——這正是 AMDP 設計上「呼叫端無感」的目標：Code-to-Data 是實作細節，不應該讓呼叫端的程式碼變複雜。

預期輸出 (SE38 執行 ZR_AM01_DEMO，實測畫面)

```
AA  American Airlines
AB  Air Berlin
AC  Air Canada
AF  Air France
AZ  Alitalia
BA  British Airways
CO  Continental Airlines
DL  Delta Airlines
FJ  Air Pacific
JL  Japan Airlines
LH  Lufthansa
NG  Lauda Air
NW  Northwest Airlines
QF  Qantas Airways
SA  South African Air.
SQ  Singapore Airlines
```

18 個航空公司代碼各出現一次——這是修正 `WHERE mandt = :iv_mandt` 之後的正確結果；沒加這個過濾條件時，同一批資料會依這套系統裝的 Client 數量重複出現。

團隊實務備註

- 這三個 **AMDP** 簽章限制 (`VALUE()`、`DEFAULT` 只能常數、無 `RETURNING/CHANGING`) 是編譯期就會擋下來的，不是執行期才發現的錯誤——語法檢查/啟用階段系統會直接指出哪一個參數有問題，照錯誤訊息修正即可，不用死背這些規則
- **USING** 子句要列出 **SQLScript** 裡實際引用到的所有資料庫物件 (這題是 `USING scarr`)，這是編譯期的靜態依賴宣告，不能在 **SQLScript** 執行期間動態決定要查哪張表 (**AMDP** 不支援真正的動態 SQL 去查一張完全沒在 **USING** 裡宣告過的表)
- **Client** 過濾這件事，往後每一題寫 **AMDP** 都要記得處理：不是這張表才有多 Client 資料的問題，是任何 client-dependent 的表 (幾乎所有業務表都是) 透過 **AMDP** 存取時，都要自己決定要不要、以及怎麼過濾 Client——這題選擇「呼叫端傳入 `iv_mandt`」，之後題目如果有更適合的做法 (例如用 **HANA** 的 `session_context`) 會再對照介紹
- **OPTIONS READ-ONLY** 這題有加、但這題的 **SQLScript** 本來就只有 `SELECT`，就算不加這個選項行為也一樣——之所以還是照加，是因為明確宣告「這個程序不會寫資料庫」是好習慣，之後遇到真的會寫入的 **AMDP Method** 時，兩者的對比才會清楚

思考題

1. 如果这套系统只有一個 Client (例如你自己的開發沙箱)，`SELECT * FROM scarr` (不加 `WHERE mandt = ...`) 還會不會有問題？ (提示：就算只有一個 Client，這個 **AMDP Method** 依然是「沒有做 Client 過濾」的寫法，只是這套特定系統的資料剛好只有一份，看不出問題——這跟「這段程式碼本身沒問題」是兩回事)
2. 這題的 `USING scarr` 只列了一張表——如果 **SQLScript** 裡用 `JOIN` 同時查詢 `SCARR` 跟 `SPFLI` 兩張表，**USING** 子句該怎麼寫？
3. `IF_AMDP_MARKER_HDB` 是一個沒有任何方法的空介面，`INTERFACES if_amdp_marker_hdb`。這一行如果拿掉，你覺得啟用會發生什麼事？ (提示：這個介面是框架用來識別「這個類別可能含有 **AMDP Method**」的標記，可以對照 **OOP op07** 學過的介面概念，思考「標記介面」(marker interface) 這種只為了打標籤、沒有任何方法要實作的介面用途)

答案

見 `zcl_am01_carriers.clas.abap`、`zr_am01_demo.prog.abap` (SAP 端物件 `ZCL_AM01_CARRIERS` / `ZR_AM01_DEMO`，套件 `$TMP`)。